

Slice Phase 8 Completion

1. Purpose and Scope of Phase 8

Phase 8 in the Slice roadmap is defined as “Minimal User Interface & Interaction.” The objective is to expose the existing Slice back-end - data, quant, intuition, and risk layers - through a small, inspectable surface that lets the PM see how the system thinks, without adding new intellectual features beyond those already specified in Phases 1–7.

In the current implementation, Phase 8 is complete when:

- The FastAPI application exposes the core Slice API and the memory sub-application.
- A Streamlit UI can be started locally and successfully renders the portfolio, strategy, diagnostics, and narrative contexts using the Phase-4/5/6/7 models.
- All context-shaping code compiles and runs even when the database is empty (no theses, no positions, no trades), returning safe placeholder structures instead of errors.
- The UI remains strictly read-only and demonstrative: it renders context objects exactly as produced by the back-end and does not attempt live trading, execution control, or new prompt/contract design.

2. FastAPI Surface (`src/slice/api/main.py`)

Phase 8 introduces a single consolidated FastAPI app that is the canonical endpoint for Slice’s HTTP surface:

- app: FastAPI(title="Slice API", version="0.1").
- Routers:
 - /session → session_router (session-oriented endpoints defined earlier in the project).
 - /intelligence → intelligence_router (analysis/LLM-facing endpoints).
- Mounted sub-application:
 - /memory → memory_app, the dedicated application that handles observation/embedding workflows from Phase 5.

The Phase 8 contract is:

- The main FastAPI object lives in `src/slice/api/main.py`.
- All future HTTP endpoints (including those used by the UI or external tools) are expected to hang off this app, either directly via routers or via mounted sub-apps.
- The API layer is thin: it delegates logic to repositories, context builders, and risk interfaces rather than re-implementing business logic in the web layer.

3. Risk Interface Stub (`src/slice/risk/interface.py`)

Phase 8 requires the rest of the system to be able to ask for a "risk snapshot" without forcing completion of the full quantitative risk engine. To satisfy this requirement, a minimal, safe interface is implemented:

- RiskSnapshot (Pydantic model):
 - book_gross: float
 - book_net: float
 - duration: Optional[float]
 - dv01: Optional[float]
 - exposures: List[Dict[str, Any]]
 - backtests: List[Dict[str, Any]]
- _select_trades(thesis_id, portfolio_id):
 - Routes selection through TradeRepository:
 - list_for_thesis(thesis_id)
 - list_for_portfolio(portfolio_id)

- list_all() fallback.
- get_snapshot(thesis_id=None, portfolio_id=None) -> Optional[RiskSnapshot]:
 - Uses _select_trades to determine whether any trades exist.
 - If no trades are present → returns None.
 - If trades exist → returns a placeholder RiskSnapshot with:
 - book_gross = 0.0
 - book_net = 0.0
 - duration = None, dv01 = None
 - exposures = []
 - backtests = []
 - All exceptions are caught and treated as "no snapshot" (returns None). The function never raises.
- render_risk_snapshot_text(snapshot: RiskSnapshot) -> str:
 - Deterministic, non-LLM textual rendering used by higher layers and the UI.
 - Formats book_gross, book_net, optional duration/dv01 and enumerates exposures and backtests if present.

This completes the Phase 8 requirement that downstream components have a stable risk interface and JSON-serializable structure, even though the quantitative risk engine is still intentionally minimal.

4. Intelligence Context DataAccess (src/slice/intelligence/context/data_access.py)

Phase 8 adds a consolidated DataAccess layer that shapes the objects sent to LLM and UI. It hides repository details and produces four key context payloads that match the Phase 6–8 design contracts.

Core traits:

- DataAccess holds references to ThesisRepository, PortfolioRepository, TradeRepository, and macro/driver utilities.
- It exposes methods that return plain dictionaries ready for JSON serialization.

Major methods:

- 1) get_all_theses() → List[Thesis]
 - Thin wrapper over ThesisRepository.list_all(), used by both portfolio and narrative views.
- 2) get_portfolio_context() → Dict
 - Returns a structure with:
 - positions: a list of current positions (or empty list).
 - totals: { "portfolio_value", "gross_exposure", "net_exposure" } with zeros when no portfolio is present.
 - risk_profile: currently an empty dict placeholder.
 - factor_exposures: { "position_factors": {}, "aggregate": { "beta_real_yield", "beta_dxy", "beta_growth" } } filled with zeros.
 - thesis_exposures: { "theses": [], "unassigned": { "weight": 0, "positions": [] } }.
 - If repositories raise or return nothing, the method still returns this stable structure.
- 3) get_strategy_context() → Dict
 - Used by the strategy/intelligence layer.
 - Shape:
 - active_theses: list of structured theses; currently empty when DB has none.
 - current_portfolio: same shape as get_portfolio_context()["portfolio"].
 - risk_profile: placeholder {}.
 - macro_view: { "macro_snapshot": { growth, inflation, rates, liquidity, fx }, "regimes":

{ growth, inflation, liquidity, usd } }.

- In the current Phase-8 build the macro_snapshot numeric fields are NULL/None.
- Regimes are set to "unknown" for each key.
- constraints: { "max_position_weight": 0.2, "max_single_name_short": 0.1 }.

4) get_portfolio_diagnostics_context() → Dict

- Focused on diagnostics and stress testing.
- Shape:
 - portfolio: same structure as in portfolio context.
 - risk_profile: {} placeholder.
 - factor_exposures: same aggregate beta structure with zeros.
 - thesis_exposures: same as in portfolio context.
 - stress_tests: a static list of scenario placeholders:
 - { "scenario": "rates_up_100bp", "impact": None }
 - { "scenario": "usd_shock", "impact": None }.
 - recent_performance: currently a placeholder {}.

5) get_narrative_context() → Dict

- Combines theses, macro_view, portfolio snapshot, quant summaries and a commentary window.
- Shape:
 - theses: list, currently empty when DB has no theses.
 - macro_view: same macro_snapshot + regimes structure as strategy context.
 - portfolio_snapshot: portfolio totals and positions as above.
 - quant_summaries: { "strategies": [], "scenarios": [], "risk_flags": [] }.
 - commentary_window: {} placeholder for future use by narrative-generation tools.

All of these context builders are intentionally conservative: they never throw, never depend on LLM responses, and always return a fully-formed dictionary with predictable keys so that Phase 9 can test and harden these contracts.

5. Streamlit UI Shell (ui/app.py)

Phase 8 introduces a minimal, read-only Streamlit front-end that surfaces the context objects built by DataAccess. It is intentionally thin and only goes as far as the Phase 8 architecture allows.

Key characteristics:

- Entry point: ui/app.py, launched with:
cd ~/dev/slice
streamlit run ui/app.py

- Data loading:

- A cached helper load_portfolio_view() builds a DataAccess instance and materializes the portfolio context and a simple depth metric.
- The same DataAccess instance is reused for strategy, diagnostics, and narrative tabs.

- Tabs / Sections (names may vary slightly in code but map to Phase-8 intent):

1) Portfolio tab:

- Renders the portfolio context as JSON, including positions, totals, factor exposures, and thesis exposures.
- Uses st.json so the user can inspect the raw structured output.

2) Strategy Context tab:

- Displays the strategy_context payload labelled "Strategy Context (Phase 8)".
- Shows active_theses (currently []), current_portfolio, risk_profile, macro_view (with NULL macro_snapshot fields and "unknown" regimes) and constraints.

3) Portfolio Diagnostics tab:

- Displays `portfolio_diagnostics_context`, including `factor_exposures`, `thesis_exposures`, `stress_tests` placeholders, and `recent_performance`.

4) Narrative Context tab:

- Displays `narrative_context`: `theses`, `macro_view`, `portfolio_snapshot`, `quant_summaries`, and `commentary_window`.

Screenshots from the running app confirm that with an empty database the UI shows:

- Concentration and factor exposure blocks with zeros and false flags.
- `Thesis_exposures` with empty "theses" and unassigned weight 0.
- Strategy/diagnostics/narrative contexts where `macro_snapshot` numeric fields are null, regimes are "unknown", and positions/strategies/scenarios lists are empty.

This behavior is expected and correct for Phase 8: it demonstrates that the context wiring is functional and stable even before real data is ingested.

6. Behavior With and Without Data

Phase 8 must behave correctly both with a blank database and with early test data.

- Empty-state guarantees:

- All context methods return total structures with zeroed numeric aggregates and empty lists.
- `get_snapshot()` returns `None`, which higher layers treat as "no risk snapshot" rather than an error.
- The Streamlit app renders consistent JSON blocks (zeros, nulls, empty arrays) instead of crashing.

- Data-present behavior (for Phase 9 to validate):

- When minimal sample theses, trades, or positions are inserted via repositories, `DataAccess` should surface them directly in the relevant contexts without any change to the UI.
- The risk interface is currently a structural stub; even when trades exist, it intentionally returns zeroed values until the full risk pipeline is implemented in later phases.

The completion criterion for Phase 8 is therefore satisfied: the slice of the system exposed to the PM via UI and API is structurally correct, robust to missing data, and aligned with the contracts defined in the architecture document.

7. Constraints and Non-Goals for Phase 8

To keep the implementation aligned with the roadmap, Phase 8 explicitly avoids:

- No new LLM prompts, analytical behaviors, or autonomy rules beyond what was already laid out in Phases 5–7.
- No trade execution controls in the UI (no buttons that place orders or alter external accounts).
- No re-design of macro driver or regime models; `macro_view` uses the "unknown" regime placeholders until the quantitative and macro rails are finalized.
- No heavy UI polish or design system: the Streamlit app is utilitarian and focused on visibility and contract validation.

Within these constraints, Phase 8 is complete: it exposes the internal state and contracts of Slice in a way that Phase 9 can now test, harden, and refine without changing the overall architecture.